

SQLD('24.03) 개정판 개념정리

작성자 : 대발대발('24.03.10),

업데이트('24.06.19)

본문 및 피드백 링크 Click ! (해당 링크에서 Notion 스크립트로 볼 시 좀 더 직관적)

▼ 준비 책 및 자료

'유선배' → 비추천.... 다시 SQLD를 준비한다면... 이기적으로 준비할 것 같음

'노랭이 개정판' → 필수! 무조건 3회 이상 돌려보고 오답 정리해서 또 풀어볼 것

https://www.youtube.com/watch?v=8uP_E6SyiuM → 메타코드 2시간 강의

[https://www.youtube.com/playlist?](https://www.youtube.com/playlist?list=PL_hDrax6_oduENm_BrLTbbzLQCWYYf)

[list=PL_hDrax6_oduENm_BrLTbbzLQCWYYf](https://www.youtube.com/playlist?list=PL_hDrax6_oduENm_BrLTbbzLQCWYYf)

→ (비개정판) 노랭이 문제 풀이 강의 Youtube → 비추천함 무조건 개정판 노랭이 구매할 것

[50회 이전 기출]

<https://yunamom.tistory.com/category/IT자격증 공부/SQLD 기출문제>

▼ 출제 문제 및 범위 유형

변경 전		⇒	변경 후	
주요항목	세부항목		주요항목	세부항목
데이터 모델링의 이해	• 데이터모델의 이해		데이터 모델링의 이해	• 데이터모델의 이해
	• 엔터티			• 엔터티
	• 속성			• 속성
	• 관계			• 관계
	• 식별자			• 식별자
데이터 모델과 성능	• 정규화와 성능		데이터 모델과 SQL	• 정규화
	• 반정규화와 성능			• 관계와 조인의 이해
	• 대용량 데이터에 따른 성능			• 모델이 표현하는 트랜잭션의 이해
	• DB 구조와 성능			• Null 속성의 이해
	• 분산 DB 데이터에 따른 성능			• 본질식별자 vs 인조식별자

- 2과목 SQL 기본 및 활용(SQLP, SQLD 공통)

변경 전		변경 후	
주요항목	세부항목	주요항목	세부항목
SQL 기본	• 관계형 데이터베이스 개요	SQL 기본	• 관계형 데이터베이스 개요
	• DDL		• SELECT 문
	• DML		• 함수
	• TCL		• WHERE 절
	• WHERE 절		• GROUP BY, HAVING 절
	• FUNTION		• ORDER BY 절
	• GROUP BY, HAVING 절		• 조인
	• ORDER BY 절		• 표준 조인
	• 조인		
SQL 활용	• 표준조인	SQL 활용	• 서브 쿼리
	• 집합연산자		• 집합 연산자
	• 계층형 질의		• 그룹 함수
	• 서브쿼리		• 윈도우 함수
	• 그룹 함수		• Top N 쿼리
	• 윈도우 함수		• 계층형 질의와 셀프 조인
	• DCL		• PIVOT 절과 UNPIVOT 절
	• 절차형 SQL		• 정규 표현식
SQL 최적화 기본 원리	• 옵티마이저와 실행계획	관리 구문	• DML
	• 인덱스 기본		• TCL
	• 조인 수행 원리		• DDL
			• DCL

출처 :한국데이터산업진흥원

▼ 1과목

▼ 데이터 모델링의 이해

데이터 모델링이란?

: 데이터 모델링은 '현실 세계'를 단순화하여 표현하는 기법

데이터 모델링 특징 및 목적

특징

• 추상화

: 현실세계, 개념을 일정한 형식으로 '간략하게' 표현

• 단순화

: 현실세계를 '정해진 표기법'으로 단순하고 쉽게 표현, 핵심에 집중 + 불필요 제거

• 명확화

: 불분명함(애매모호함)을 제거하고, '정확하게' 현상을 기술

목적

- 단순히 DB, 시스템 만을 구축하기 위한 것이 아닌 업무 설명, 분석, 형상화 목적도 있음
 - 분석된 모델로 실제 DB 생성하며 개발 및 데이터 관리에도 사용
-

데이터 모델링 유의점 및 3가지 관점 및 중요 3요소

유의점

- 중복(Duplication)
: 같은 데이터가 엔티티에 중복 저장되면 안된다.
- 비유연성(Inflexibility)
: 애플리케이션의 '사소한 변경'에도 데이터 모델이 수시로 변경되면 안된다.
→ 데이터 모델과 프로세스 분리해서 유연성 높여야한다.
- 비일관성(Inconsistency)
: 중복이 없는 경우에도 비일관성 발생 가능성 있음
→ 데이터 간의 연관 관계에 대해 명확하게 정의

관점

- 데이터 관점 (What, Data)
: 어떤 데이터들이 업무와 얹혀있는지
- 프로세스 관점 (How, Process)
: 업무가 실제로 처리하고 있는 일이 무엇인지
- 데이터와 프로세스의 상관 관점 (Data vs Process, Intercation)
: 프로세스 흐름에 따라 데이터가 어떤 영향을 받는지

중요 요소

- Things : 대상(Entity)
 - Attribute : 속성
 - Relationships : 관계
-

모델링의 3가지 단계

1. 개념적 데이터 모델링

: '전사적'으로 수행, 업무 중심적이고 포괄적인 수준의 모델링(추상화 레벨 가장 높음)

2. 논리적 데이터 모델링

: Key, 속성, 관계들을 표현하는 단계 → 정규화 활동이 이루어지는 단계

: 논리데이터모델을 대상으로 정규화를 하는 것

3. 물리적 데이터 모델링

: 실제 DB를 구현할 수 있도록 성능, 가용성등 물리적 요소 고려하는 단계

데이터 스키마 단계에 따른 독립성

스키마란?

: 테이블이 어떠한 구성으로 되어있는지, 어떤 정보를 가지고 있는지에 대한 기본적인 테이블의 구조를 정의한 것

데이터 스키마의 구조

| USER

- 외부 스키마 : 각(여러) 사용자가 보는 스키마 정의 및 표현
- 개념 스키마 : 모든(여러X) 사용자가 보는 데이터 정의 및 표현 & 관계를 정의하는 단계
- 내부 스키마 : 물리적인 저장 구조를 나타내는 단계 → 저장 구조, 칼럼, 인덱스 정의

| DB

** 논리적 독립성 : 개념 스키마가 변경 되어도 외부 스키마는 영향 X 외부 - 개념

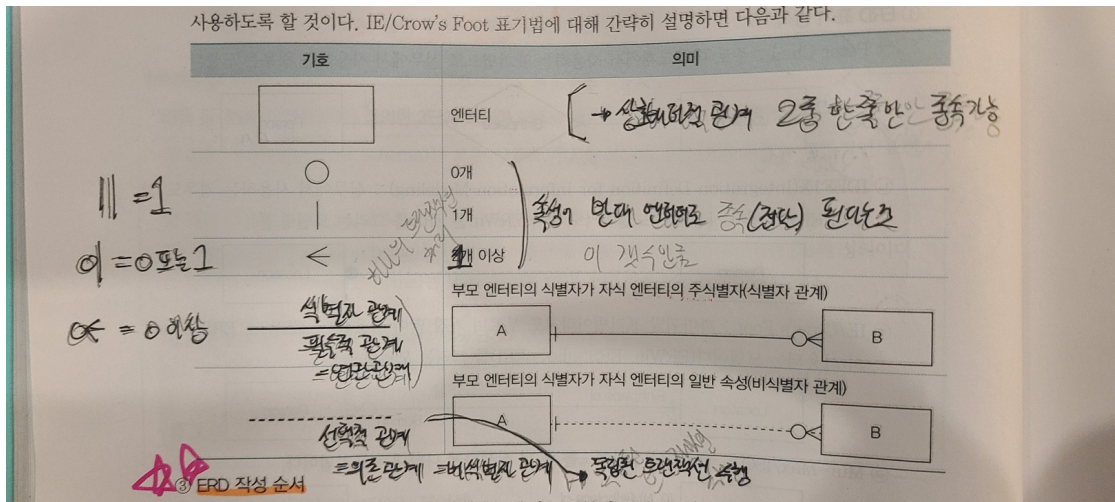
** 물리적 독립성 : 내부 스키마가 변경 되어도 개념/외부 스키마는 영향 X 외부, 개념 - 내부

ERD 작성 순서

1. 엔티티 도출
2. 엔티티 배치
3. 엔티티 관계 설정
4. 관계명 기입

5. 관계 참여도 기입

6. 관계 필수/선택 여부 기입



- Entity는 네 부분의 모서리가 둥근 형태인 Soft-Box로 표현
- Entity 이름은 Box 내부 상단에 표시
- Attribute 중 필수로 값을 입력하며 식별자인 속성은
 - # (Mandatory)를 표시
- Attribute 중 필수로 값을 입력하여야 하는 속성은
 - * (Mandatory)를 표시
- Attribute 중 선택적인 입력을 하여야 하는 속성은
 - o (Optional)를 표시

엔티티란? 엔티티의 특징

: 업무에서 쓰이는 데이터들을 용도별로 분류한 데이터의 그룹 = 엔티티

엔티티의 특징

- 업무에서 쓰이는 정보여야 함
- 식별자가 있어야함
- 2개 이상의 인스턴스 가져야함
- 반드시 속성 가져야함 → 이 때 하나의 인스턴스는 2개 이상의 속성을 가짐
→ 즉 하나의 엔티티는 2개 이상의 속성을 가짐

- 다른 엔티티와 1개 이상의 관계
-

엔티티 분류 방법과 그에 따른 종류

유형, 무형에 따른 분류 → 개사유~ 계서유~

- 유형 엔티티 : 모델링 대상이 **물리적인 형태가 존재** ex) 상품, 회원
- 개념 엔티티 : 모델링 대상이 **형태 없음** ex) 부서, 학과
- 사건 엔티티 : 모델링 대상이 **행위로 인해 발생하는 것** ex) 주문, 이벤트 응모

발생 시점에 따른 분류 → 행기중 !

- 기본 엔티티
: 모델링 대상이 업무에 대해 원래 존재하는 요소 → 독립적, 자식 엔티티 가질 수 있음
ex) 상품, 회원, 부서
 - 중심 엔티티
: 모델링 대상의 업무 과정 중 하나, 기본 엔티티로부터 파생, 행위 엔티티 생성
ex) 주문, 매출, 계약
 - 행위 엔티티
: 2개 이상의 엔티티로부터 파생
ex) 주문 내역, 이벤트 응모 이력 등
-

엔티티 명명 주의점

- 업무에서 실제 쓰이는 용어 사용
 - 한글 약어 사용X, 영어 대문자로 표시
 - 단수 명사로 표현, 띄어쓰기 X
 - 의미상 중복 X(주문, 결제 엔티티는 중복 가능)
 - 명확하게 표현
-

속성이란, 속성의 특징

: 엔티티의 특징을 나타내는 최소의 데이터 단위

속성의 특징

- 더 이상 쪼개지지 않는 레벨
- 업무에서 필요로 하는 항목
- 엔티티를 설명, 인스턴스를 설명
- 하나의 속성은 하나의 속성값만 가짐 → 여러개 가지면 1차 정규화
- 일반 속성은 정해진 주식별자에 함수적 종속성 가져야한다
→ 완전 함수적 종속이 아닌 부분 종속이면 2차 정규화 해준다.

ex) PK가 2개의 속성으로 이루어져있는데 {속성1, 속성2} 에서 속성 2에만 종속성 가지면 2차 정규화로 엔티티 추가 생성해서 각 엔티티마다 완전 함수적 종속 충족시켜줌

속성의 특성에 따른 분류

일반적인 특성에 따라 분류

- 기본 속성
: 업무 프로세스(기본 틀) 분석했더니 바로 정의 가능한 속성
- 설계 속성 - 인스턴스에 유니크함을 부여하는 속성(PK의 토대)
: 업무엔 없으나, 모델링 하다보니 고유함 보전하기 위해 필요해져서 만들어짐
ex) 학번, 사번 등등
- 파생 속성 → 그냥 파생 들어가면 다 성능, 편의 위해 새로 만든 엔티티의 속성
: 데이터를 조회할 때 빠른 성능 낼 수 있도록 원래 속성값을 계산하여 저장할 수 있도록 하는 속성 ex) 평균, 재고 등등..
→ 데이터 정합성 고려 & 가급적 적게 정의

구성 방식(각 속성 및 엔티티와의 관계)에 따른 분류

- PK 속성
: 인스턴스의 유니크함을 부여하는 속성, 일반 속성들의 종속성을 가진 키
(기본키, 주식별자 키) #으로 표현 ex) 학번, 사번
- FK 속성
: 다른 엔티티에서 가져온 속성(외래키), 다른 엔티티와의 관계를 맺게 해줌
→ 주식별자에 있는 속성이 FK가 될 수 있음 ex) #사원번호(FK)
ex) 학과 코드, 회원 등급 코드, 부서 코드 → 학과에 따른 엔티티가 있겠지? 개량 연결

- 일반 속성 : PK, FK를 제외한 나머지 속성

속성의 분해 가능 여부에 따른 분류

- **단일** 속성 : 속성이 하나의 의미로 구성
- **복합** 속성 : 여러개의 의미로 구성(주소 = 시+구+동)
- **다중값** 속성 : 속성이 여러개 값 가짐 → 1차 정규화 or 별도 엔티티 생성

속성이 만들어낸 데이터 모델의 개념

- 도메인
: 속성이 가질 수 있는 속성 값의 범위
- 용어 사전
: 속성의 **이름을 정확, 직관적으로 부여하기 위한 용어 사전**
- 시스템 카탈로그
: 시스템 자체에 관련있는 데이터를 가진 DB
: 시스템 테이블로 구성 & **SQL로 조회 가능**
: 여기 저장된 데이터 = **메타 데이터**, SELECT만 가능 INSERT, UPDATE 등등 불가능

관계란? 관계의 종류

: 엔티티와 엔티티 사이에 속성끼리의 연결에 의해 만들어지는 상관 관계

종류

- 존재 관계 : 모델링 된 엔티티들이 존재로서 관계를 가짐
- 행위 관계 : 모델링 된 엔티티들이 행위에 의해 관계를 가짐

UML의 클래스 다이어그램에 의해 나뉘는 종류

- 연관 관계
: 필수적 관계(존재적 관계, 식별자 관계) - 항상 서로 이용(실선)
: **멤버 변수로 선언**
- 의존 관계
: 선택적 관계(비식별자 관계) - 상대 클래스 행위에 따라 이용(점선)
: **행위 코드 오퍼레이션에서 파라미터로 사용**

관계 표기 방법(ERD)에 따른 특성 분류

- 관계명
: 관계 이름은 시작 엔티티 - 능동적/끝 엔티티 - 수동적 '동사' 사용
- 관계 차수
: 각 엔티티 끼리의 관계에 참여하는 '속성의 수' 1:1, 1:M, M:N 형식으로 구분
- 관계 선택 사양
: 필수적 관계(엔티티끼리 항상 관계), 선택적 관계(행위에 의해 관계 여부가 성립)
ex) 한 수업 엔티티에 참여 엔티티, 과제 엔티티가 있으면
참여는 수업이 있을 때 마다 항상 관계가 성립되어서 조회가 되지만
과제는 과제가 있는 날에만 관계를 맺고 조회가 되기 때문에 이러한걸 구분

관계 체크 사항(두 엔티티 사이 관계 정의 시 유의할 사항)

- 두 엔티티 사이 관심있는 연관 규칙이 존재하는가
- 두 엔티티 사이 정보의 조합이 발생하는가
- 업무 기술 시, 장표의 관계 연결을 가능하게하는 동사가 있는가
- 업무 기술 시, 장표의 관계 연결을 가능하게하는 규칙이 서술 되어 있는가

식별자란? 주 식별자의 특성

: 각각의 인스턴스를 구분 가능하게 만들어주는 대표 속성을 뜻한다.

주 식별자란? 주 식별자의 특성 # 으로 표현

: 주 식별자는 PK(Primary Key)에 해당 하는 속성 → PK는 여러개 존재 할 수 있음

- 유일성: 해당 속성이 인스턴스를 유일하게 식별할 수 있는 성질을 가졌는지
- 최소성: 최소한의 속성들로만 유일성을 보장하게 하는지
- 불변성: 속성값이 변하지 않아야함
- 존재성: 속성값은 NULL이 될 수 없음
→ ex) 유일성과 최소성을 만족하는 속성은 보조키로서 존재할 수 있다.
→ 즉 특정 특성을 만족함에 따라 속성은 특정 키로서 존재가능

식별자의 특성과 특정 여부에 따른 분류

대표성 여부

- 주 식별자(PK) - # 으로 표현

: 유일성, 최소성, 불변성, 존재성을 모두 만족하는 식별자

→ PK는 여러 속성이 존재 할 수 있으나, 여러 속성이 존재 할 경우 나머지 일반 속성들이 해당 PK들 속성들에 대해 함수적 종속성을 띄어야함 → 그렇지 않으면 2차 정규화하여 부분 종속에 해당하는 속성들만 따로 추가 엔티티를 생성한다.

- 주 식별자 도출 기준

- 해당 업무에서 자주 이용되는 속성
- 명칭, 내역 등의 이름은 피함
- 속성 수를 최대한 적게 구성
- 자주 변하지 않는 값

- 보조 식별자

: 인스턴스 식별은 가능하나 엔티티를 대표하는 식별자는 아님

→ 즉 다른 엔티티와의 참조 관계로 연결되지 않는다.

ex) 회원 엔티티에서

#회원번호

*회원명

*아이디

→ 에서 아이디는 다른 인스턴스랑 중복될 수 없기 때문에 해당 엔티티에서 인스턴스를 구분짓게 할 수 있는 식별자이나

→ 이게 엔티티를 대표하지는 못 함

스스로 생성 되었는가에 대한 여부

- 내부 식별자

: 다른 엔티티 참조 없이 해당 엔티티 내부에서 스스로 생성된 식별자

- 외부 식별자

: 다른 엔티티에서 온 식별자 - 다른 엔티티와 연결고리 역할

→ 만약 부모 엔티티의 FK를 받아서 이를 주식별자로 사용하면

→ 해당 자식 엔티티의 PK는 **SQL 조인에서 반드시 사용**되고 WHERE 절에서 사용 가능성이 높음

단일 속성인지에 대한 여부(주 식별자 구성이 여러 속성인가)

- **단일** 식별자 : 주 식별자가 1개의 속성으로 구성
- **복합** 식별자 : 주 식별자가 **2개 이상의 속성**으로 구성
 - 주 식별자가 2개 이상이면 해당 속성들의 우선순위를 잘 매겨서 잘 복합시킨 후 일반 속성들에게 종속시켜야 주 식별자로서 기능을 다 하게 된다.

대체되었는지 기존에 있는지에 대한 분류

- **원조(본질)** 식별자 : **업무에 의해 만들어지는 식별자**, 가공되지 않은 원래 식별자
- **인조(대리)** 식별자 : 인위적으로 만들어지는 식별자, 주 식별자가 복잡할 때 이를 통합
 - ex) 주문번호 - 대표적 인조, 대리 식별자
 - 기존 : 사번+주문일자+순번을 주 식별자로 두고 주문을 처리하다가
 - 이를 "주문번호" 라는 단일 속성의 주 식별자로 만들면 이게 인조, 대리 식별자가 됨

식별자 관계 vs 비식별자 관계

식별자 관계

→ 트랜잭션에 의한 관계 - 동시에 커밋, 롤백 - 하나의 커밋 단위로 엔티티들이 묶임

: 부모 엔티티의 식별자 속성이 **자식 엔티티의 주 식별자**가 되는 관계

- 강한 연결 관계
- 실선(항시 연결)
- 부모-자식 관계가 항시 유지
- **SQL문의 조인을 최소화 해줌**

비식별자 관계

: 부모 엔티티의 식별자 속성이 **자식 엔티티의 일반 속성**이 되는 관계

- 약한 연결 관계
- 점선(선택적 연결)
- 부모-자식 관계가 유지 안 될 수 있음

→ 일반 속성 값은 NULL이 들어갈 수 있기 때문에 부모 엔티티의 식별자 속성에 값이 없을 때 자식 엔티티의 속성 값(인스턴스)이 생성 가능하다.

▼ 데이터 모델과 SQL

성능 데이터 모델링의 개요

1. 성능 데이터 모델링의 정의

- 성능 저하의 원인 중 하나는 데이터 모델링의 근본적인 디자인이 잘못되어 있는 경우도 많다
- 따라서 성능 데이터 모델링을 통해 성능향상을 도모해야한다
- 성능 데이터 모델링이란?
 - 데이터베이스 성능향상을 목적으로 설계단계의 데이터모델링 때부터 성능과 관련된 사항이 모델링에 반영될 수 있도록 하는 것

2. 성능 데이터 모델링 수행시점

- 사전에 성능 모델링을 할수록 성능 향상을 위한 비용은 적게 든다
- 분석/설계 단계에서 성능을 고려해 데이터 모델링을 수행할 경우 재업무 비용을 최소화할 수 있다
- 따라서 분석/설계 단계에서 처리성능을 향상시킬 방법을 고려해야한다

3. 성능 데이터 모델링 고려사항

- 성능 데이터 모델링 프로세스
 - 정규화 → 정규화가 1등
 - DB 용량 산정
 - 트랜잭션의 유형 파악 → 테이블 수직 분할 할 때(반정규화)
 - 용량과 트랜잭션의 유형에 따라 반정규화
 - 이력모델 조정, PK/FK 조정, 슈퍼타입/서브타입 조정
 - 성능관점에서 데이터 모델을 검증

▼ 정규화

정규화란?

: 엔티티를 작은 단위로 분리하는 과정

→ 큰 엔티티를 작은 엔티티들로 분리하고 관계 맺음

: **논리 데이터 모델**에서 행하는 과정이다.(개념 모델링 X, 물리 모델링 X)

정규화의 특징 및 하는 이유와 개념 = 장점

- 데이터의 무결성을 위해 수행
- 최소한의 데이터만을 하나의 엔티티에 넣는 과정, 데이터 분해 과정
- 데이터 **일관성 확보**
- 데이터 **독립성 확보** → 데이터 중복 제거
- 데이터 **유연성 확보** → 필요 데이터들의 분할로 인해 유연하게 접근 가능
- **입력, 수정, 삭제 성능은 일반적으로 향상**
 - → 조회 성능이 저하 될 수 있음

정규화의 단점

- 엔티티 갯수 증가
- 이로 인한 관계 증가
- 데이터 조회 시 여러번의 조인이 요구
- **조회 성능의 저하**
 - 식별자, 비식별자랑 헷갈리지말자 → 식별자 = join 최소화

정규화의 종류

각 정규화를 통해 이루어지는 행위가 있는데

이 행위를 만족하는 엔티티 구조는

제 1 정규형 릴레이션

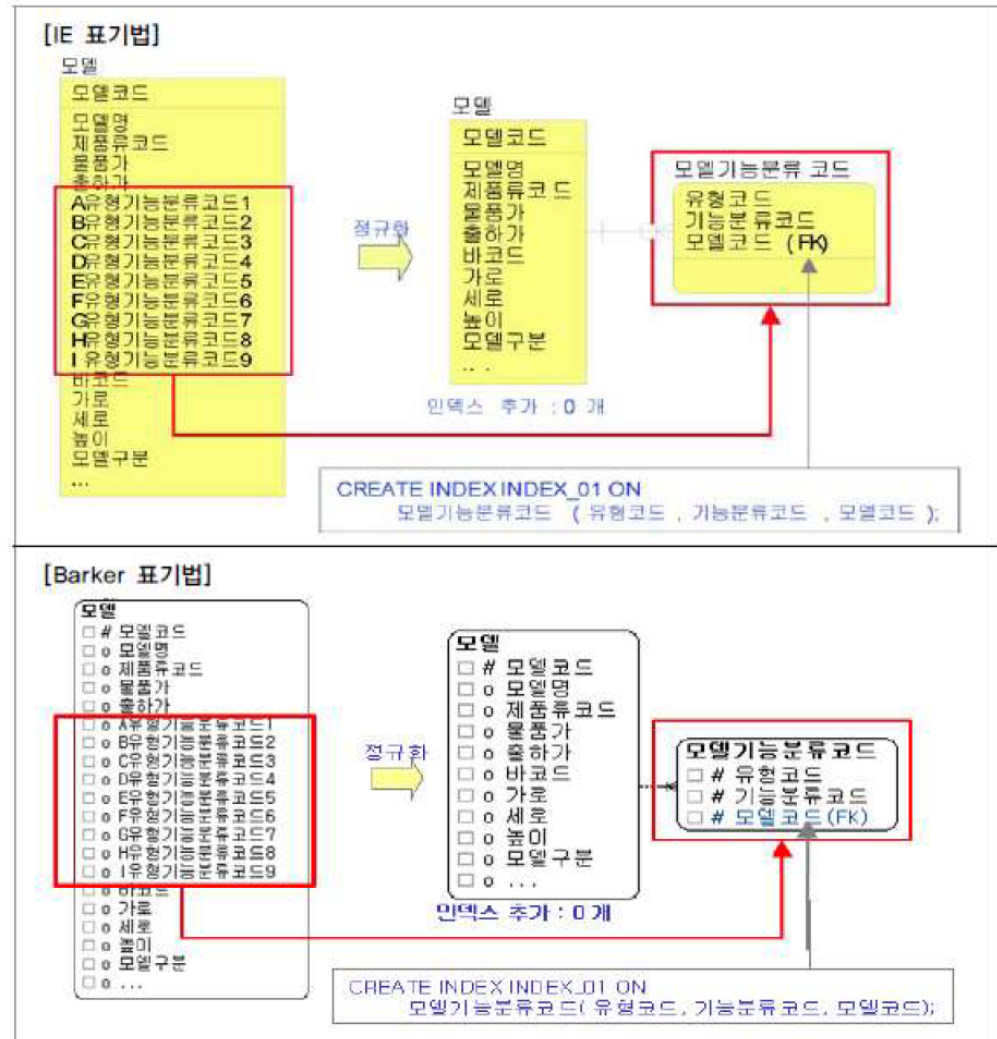
제 2 정규형 릴레이션

제 3 정규형 릴레이션 이라고 칭한다.

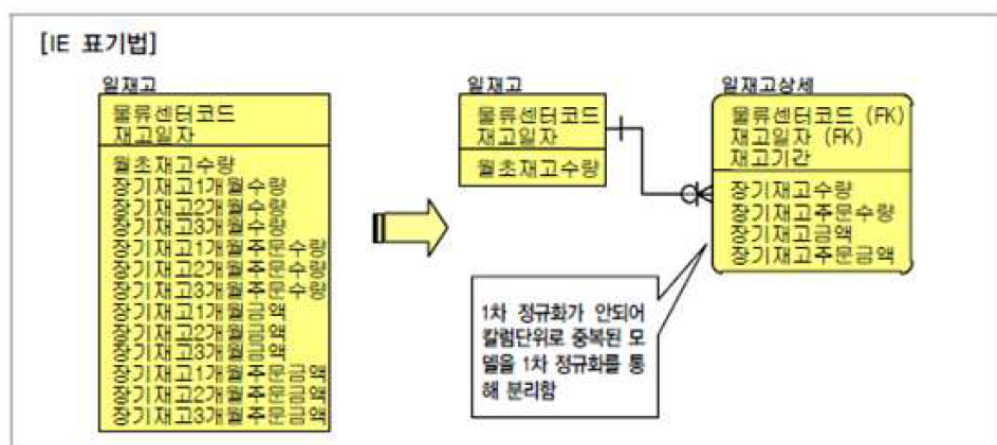
▼ 제 1 정규화

: 테이블 칼럼들이 원자성(특성의 중복을 방지) 갖게 하기 위해 엔티티 분해

→ 하나의 인스턴스가 비슷한 속성을 여러개 가지지 않게 하기 위해 분리하는 것



[그림 1-2-9] 정규화된 데이터 모델



▼ 제 2 정규화

: 엔티티의 모든 일반 속성은 반드시 주 식별자의 모든 속성들에

'부분 종속'이 아닌 '완전 종속'을 가져야한다.

이 때 만약 '부분 종속'을 가지는 일반 속성이 있다면 해당 속성과 해당 속성의 결정자인 부분 종속을 이루고 있는 주 식별자의 속성을 따로 떼어내 추가적인 엔티티를 만들어 제 2 정규형을 만족하는 릴레이션을 구축하는 것

: 또는 주 식별자의 속성이 아닌 일반 속성 끼리 종속 관계를 맺어도 이에 대해 해당 일반 속성이 새로운 엔티티에서 제 2 정규성을 만족하도록 엔티티를 추가적으로 만들어준다.

ex) 엔티티 1에서 일반속성A→일반속성B 이면

일반속성 B는 엔티티 1에서 제거하고 A는 엔티티 1에 남겨둔 채로

엔티티 2를 만들고 일반속성 B를 엔티티 2의 일반속성으로 엔티티 1의 일반속성 A를 FK로 사용하여 엔티티 2의 주식별자로서 엔티티를 구축하고 릴레이션을 유지하게하는 것

▼ 제 3 정규화

: 정규화된 엔티티의 일반 속성들은 주 식별자에만 함수적 종속을 가져야한다.

그런데 만약 주 식별자의 속성들끼리 종속 관계를 가지고 그 이후에 또 일반속성에 대해 결정자가 되던지 일반 속성끼리 종속성을 가지는데 이 때의 결정자가 주 식별자 속성에 종속되어있는등

$A \rightarrow B, B \rightarrow C$ 와 같은 '이행적 종속' 을 이루는 TABLE(엔티티) 일 때

이러한 '이행적 종속'을 깨도록 추가적인 엔티티를 만들고 관계를 형성해주는 것이

제 3 정규화 이다.

- BCNF 정규화

: 모든 결정자가 후보키가 되도록 테이블을 분해하는 것

→ 후보키 : 식별자의 '유일성', '최소성' 을 만족하는 속성 집합(or 단일속성)

- 제 4 정규화

: 여러 칼럼이 하나의 칼럼 종속시킬 때 분해해서 '다중값 종속성' 제거

- 제 5 정규화

: 조인에 의해 새로운 종속성 발생 시 이를 막기 위해 엔티티 재 분해

▼ 반정규화

반정규화란? 특징과 하는 이유

: '정규화 된' 데이터 모델(엔티티, 속성, 관계)에 대해

'성능 향상', '개발, 운영의 단순화' 를 위해 데이터를 중복, 통합, 분리 하는 기법

: 정규화 시 엔티티 갯수 증가, 관계 증가 → 여러 조인 요구

→ 이런 경우 디스크 I/O 양이 많아져 성능이 저하되거나 경로가 멀어서 '조인'으로 인한 성능 저하가 예상

→ 비정규화 = 정규화를 하지 않음, 반정규화 = 위를 하는 것

반정규화의 특징

- 조회(SELECT) 속도 향상
- 데이터 모델의 유연성은 저하
- → 입력/수정/삭제 성능 저하

반정규화 하는 경우

- 정규화를 통해 엔티티, 관계 수가 많아져서 조회 시 '조인'으로 인한 성능 저하 예상될 때
- 칼럼을 계산하고, 읽을 때 FK라서 여러 조인을 또 불러와서 성능이 저하 될 때

→ 즉 조인으로 인한 I/O 양이 너무 많아져서 처리 성능이 저하 될 때

→ 중복성을 증가시켜 조회 성능을 향상시킨다.

반정규화 안하면 발생하는 문제

- 성능 저하된 DB 생성
- 구축, 시험 단계에서 수정에 따른 노력 비용 발생

테이블을 가지고 반정규화 방법(병합, 분할, 추가)

테이블 병합

- 1:1 관계 테이블 병합
- 1:M 관계 테이블 병합
- 슈퍼 서브 타입 테이블 병합
 - 공통 속성과 개별 속성을 별도로 관리하는 설계 타입

테이블 분할

- 테이블 수직 분할(속성 분할)

: 트랜잭션 처리 유형 파악이 필요 → 반정규화에서 테이블 수직 분할 할 때 필요

→ 테이블 속성 개수 많을 때, 조회 성능 향상을 위해

→ 자주 쓰이는 속성을 수직 분할 → 이후 1:1 관계 이루게 된다.

- 테이블 수평 분할(인스턴스 분할, 파티셔닝)

: 물리적으로 데이터 분리

테이블 추가

- 중복 테이블 추가

: 동일한 테이블 구조 중복, 원격 조인 제거

- 통계 테이블 추가

: SUM, AVG 등 전용 테이블 추가

- 이력 테이블 추가

: 마스터 테이블의 레코드를 끊어서 테이블 추가 생성

- 부분 테이블 추가

: 이용 빈도 높은 칼럼을 복사하여 별도 테이블 생성, 물리적 디스크 I/O 줄이기 위해

칼럼을 통해 반정규화 하는 방법

- 중복 칼럼 추가 → 중복 추가는 다 JOIN 감소 시키기 위해(중복 테이블 추가)

: 조인 감소를 위해 중복 칼럼 추가

ex) 최근 상품 가격

- 파생 칼럼 추가 → 파생 속성이 이걸 뜻하는 것 → 부하 줄이기

: 미리 값을 계산하여 칼럼에 보관

- 이력 테이블 칼럼 추가

: 대량의 이력 데이터를 처리할 때 기능성 칼럼(최근값 여부, 시작&종료일 등)을 추가

- PK에 의한 칼럼 추가

: 여러 칼럼으로 이루어진 PK를 가진 테이블을 조인할 경우 단순성을 위해서
인공키를 PK로 지정하고 활용

- 응용 시스템 오작동을 위한 이전 데이터 보관 칼럼 추가

: 이전 데이터를 임시적으로 중복하여 보관

관계를 통해 반정규화 하는 방법

중복 관계 추가 방법

- 여러 경로를 거쳐 조인 할 수 있지만, 성능 저하를 예방하기 위해 추가적인 관계를 맺음

→ 중복 관계 추가는 데이터 무결성을 깨뜨릴 위험성이 없음

→ 이에 무결성을 지키면서 처리 성능을 향상 시킬 수 있음

▼ 관계와 조인

관계란?

: 부모 엔티티의 식별자를 자식에 상속하고, 상속된 속성을 매핑키(조인키)로 활용

관계의 분류

- 존재 관계
- 행위 관계

조인이란?

: 데이터 중복을 피하기 위해 테이블은 정규화에 의해 분리

→ 이렇게 분리된 테이블을 동시에 출력하거나 관계가 있는 테이블 참조 위해서는 테이블 연결

→ 이 때 이러한 연결 과정을 JOIN 이라 칭한다.

계층형 데이터 모델

: 하나의 엔티티 내에서 인스턴스 끼리 계층 구조를 가지는 경우

→ 계층 구조를 갖는 인스턴스끼리 연결하는 조인을 셀프 조인이라 한다.

(같은 테이블 내에서 여러 번 조인 되는 것)

ex) 인스턴스 A를 굶었는데 그 안에 속성 B에 대한 값이 해당 엔티티 내의 다른 인스턴스에 있는 값이여서 이들을 두 속성을 같이 SELECT 하고, WHERE &

AND로 조건 먹인다음 쿼어낼 때 두 SELECT에 의해 하나의 엔티티 내에서 여러 번 조인이 발생

상호배타적 관계

: 하나의 부모가 2개의 자식 엔티티를 가질 때 행위 조건에 따라 두 자식 중 하나의 자식만 관계를 가질 수 있는 것을 상호배타적 관계라 칭한다.

▼ 트랜잭션이란?

트랜잭션의 특징

- 하나의 연속적인 업무 단위를 뜻 함
- 트랜잭션에 묶인 엔티티들은 '필수적 관계' 가짐
- 하나의 트랜잭션에 속한 동작들은 모두 성공하거나, 모두 취소(UNDO)되어야 한다.
→ 트랜잭션의 '원자성'
- 서로 독립적으로 업무가 발생하면 안됨, 순차적으로 함
- 부분 커밋 불가, 동시 커밋&롤백

▼ 본질 식별자와 인조 식별자

원조(본질) 식별자

: 업무에 의해 만들어지는 식별자(꼭 필요한 식별자)

인조(대리) 식별자

: 원조 식별자가 PK 2개 이상인 복합 식별자 일 때

속성들을 하나의 속성으로 묶어서 사용하면 이것이 인조 식별자

: 꼭 필요하진 않지만 편의성을 위해 인위적으로 만들어지는 것

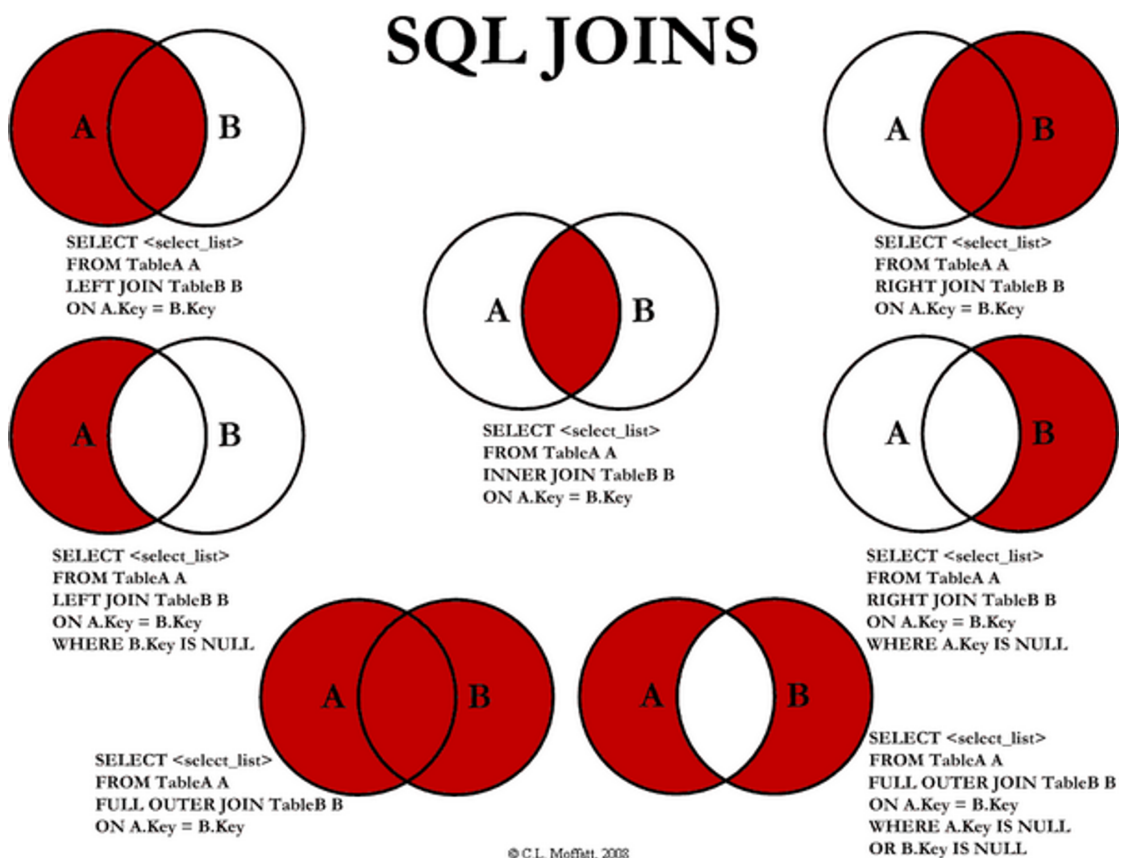
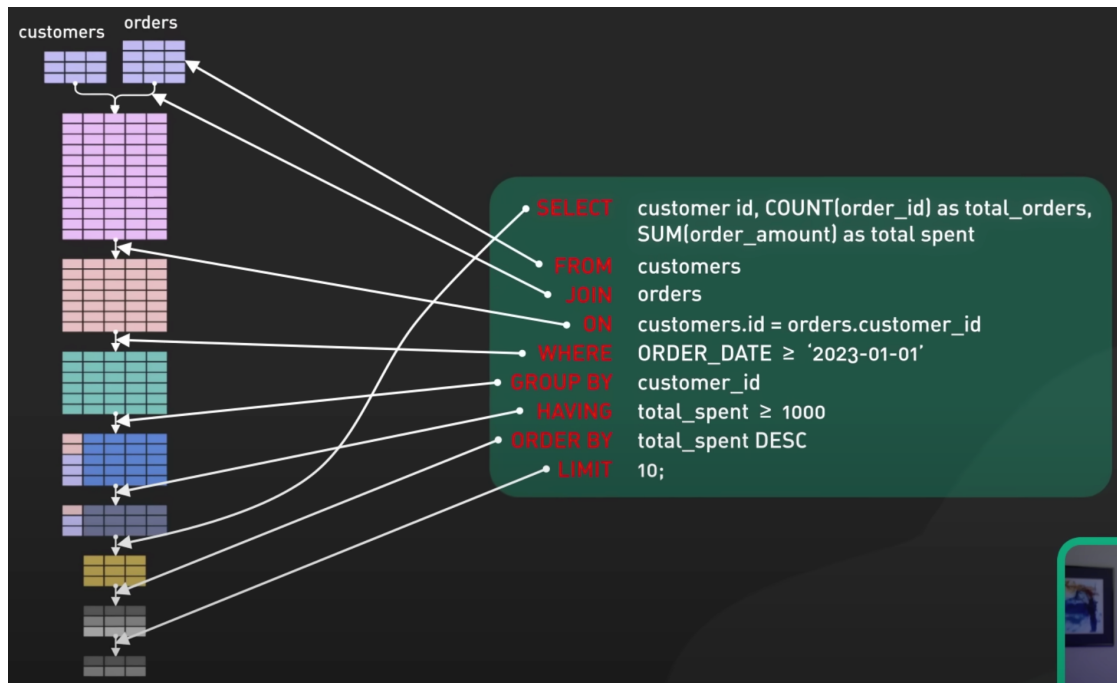
인조 식별자의 단점

- **중복 데이터 발생 가능성** → 데이터 품질 저하
- **불필요한 인덱스 생성** → 저장 공간 낭비 및 DML 성능 저하
- **개발 편의성이 줄어들 수 있음**

▼ 2과목 SQL 기본 및 활용

▼ SELECT 절의 구조와 순서

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY 순서



▼ WHERE절 특징

WHERE 절에서 함수를 사용하는 것은 가능하지만, 집계 함수(**SUM**, **AVG**, **COUNT** 등)를 직접 **WHERE** 절에 사용하는 것은 허용되지 않습니다. 이는 **WHERE** 절이 각 행을 개별적으로 평가하고 필터링하는데 사용되기 때문입니다. 집계 함수는 여러 행의 데이터를

요약하여 하나의 결과를 생성하는데, 이러한 연산은 **WHERE** 절이 처리되기 전에 개별 행을 대상으로 조건을 적용할 때는 수행할 수 없습니다.

예를 들어, **WHERE SUM(SALA) > 20000** 은 잘못된 사용 예입니다. 여기서 **SUM(SALA)** 는 여러 행의 **SALA** 값을 합산하는데, 이는 **WHERE** 절에서 수행할 수 있는 개별 행에 대한 조건이 아닙니다.

이런 조건을 적용하고자 할 때는 **HAVING** 절을 사용해야 합니다. **HAVING** 절은 **GROUP BY** 절과 함께 사용되어, 그룹화된 결과에 대해 조건을 적용합니다. 예를 들어, 총합이 20000을 초과하는 그룹을 찾고자 한다면, 쿼리는 다음과 같이 작성됩니다:

```
SELECT department, SUM(SALA) AS total_salary
FROM employees
GROUP BY department
HAVING SUM(SALA) > 20000;
```

이 쿼리는 **employees** 테이블에서 **department** 별로 **SALA** 의 합을 계산하고, 그 합이 20000을 초과하는 부서만 선택합니다. **HAVING** 절은 **GROUP BY** 로 생성된 그룹에 대해 조건을 적용하기 때문에, 여기서는 집계 함수의 결과에 조건을 적용하는 것이 적절합니다.

▼ 3장 관리 구문

구분	종류
DDL (Data Definition Language)	CREATE, ALTER, DROP, TRUNCATE
DML (Data Manipulation Language)	INSERT, DELETE, UPDATE, MERGE
DCL (Data Control Language)	GRANT, REVOKE
TCL (Transaction Control Language)	COMMIT, ROLLBACK
DQL (Data Query Language)	SELECT

명령어의 종류	명령어	설명
데이터 조작어 (DML: Data Manipulation Language)	SELECT	데이터베이스에 들어 있는 데이터를 조회하거나 검색하기 위한 명령어를 말하는 것으로 RETRIEVE 라고도 한다.
	INSERT UPDATE DELETE	데이터베이스의 테이블에 들어 있는 데이터에 변형을 가하는 종류의 명령어들을 말한다. 예를 들어 데이터를 테이블에 새로운 행을 집어넣거나, 원하지 않는 데이터를 삭제하거나 수정하는 것들의 명령어들을 DML이라고 부른다.
데이터 정의어 (DDL: Data Definition Language)	CREATE ALTER DROP RENAME	테이블과 같은 데이터 구조를 정의하는데 사용되는 명령어들로 그러한 구조를 생성하거나 변경하거나 삭제하거나 이름을 바꾸는 데이터 구조와 관련된 명령어들을 DDL이라고 부른다.
데이터 제어어 (DCL: Data Control Language)	GRANT REVOKE	데이터베이스에 접근하고 객체들을 사용하도록 권한을 주고 회수하는 명령어를 DCL이라고 부른다.
트랜잭션 제어어 (TCL: Transaction Control Language)	COMMIT ROLLBACK	논리적인 작업의 단위를 묶어서 DML에 의해 조작된 결과를 작업단위(트랜잭션) 별로 제어하는 명령어를 말한다.

- DDL에 TRUNCATE 추가
- TCL에 SAVEPOINT 추가
- DML에 MERGE 추가

▼ 3장 관리 구문 정리

▼ 전반적인 용어 정리(제약조건, 디폴트)

DEFAULT란?

: 특정 칼럼에 값이 생략되어 INSERT 될 경우 자동으로 부여되는 값

: 기본 = NULL, ALTER로 설정하면 이전의 Column data엔 적용 x

: DEFAULT 임의 수정 시 수정 이후에 생성된 칼럼만 다시 적용

제약 조건 종류

- PK
 - : NULL값 입력X
 - : 자동으로 UNIQUE 인덱스로 생성
 - : 한 테이블에 1개만 가능 → 즉 PK랑 주식별자는 다른 것(주식별자 안에 PK가 있음)
- UNIQUE
 - : NULL값 허용
 - : 주식별자 중 하나

- FK
- NOT NULL
- CHECK

DELETE, DROP, TRUNCATE 차이점

DELETE - 사용자 커밋

- 데이터 일부 또는 전체 삭제
- 롤백 가능
- UNDO 데이터 생성 → 느림

DROP - AUTO 커밋

- 데이터와 구조를 동시 삭제
- 즉시 반영
- 롤백 불가능

TRUNCATE - AUTO 커밋

- 데이터만 초기화, 구조는 가만히 둬
- 즉시 반영
- 롤백 불가능
- UNDO 데이터 생성 X → DELETE보다 빠름

- DELETE : 데이터 일부 또는 전체 삭제 → 롤백 가능
- DROP : 데이터와 구조를 동시 삭제, 즉시 반영 → 롤백 불가능
- TRUNCATE : 데이터만 초기화, 구조는 가만히 둬, 즉시 반영 → 롤백 불가능

▼ DDL (Data Definition Language) → 데이터 구조를 정의(완전 AUTO)

▼ 제약 조건 설명

The screenshot shows the Oracle SQL Developer interface with the 'EMPLOYEES' table selected. The 'Constraints' tab is active, displaying a list of 10 constraints for the table. The constraints are as follows:

CONSTRAINT_NAME	CONSTRAINT_TYPE	SEARCH_CONDITION	R_OWNER
1 EMP_DEPT_FK	Foreign_Key	(null)	HR
2 EMP_EMAIL_NN	Check	"EMAIL" IS NOT NULL	(null)
3 EMP_EMAIL_UK	Unique	(null)	(null)
4 EMP_EMP_ID_PK	Primary_Key	(null)	(null)
5 EMP_HIRE_DATE_NN	Check	"HIRE_DATE" IS NOT NULL	(null)
6 EMP_JOB_FK	Foreign_Key	(null)	HR
7 EMP_JOB_NN	Check	"JOB_ID" IS NOT NULL	(null)
8 EMP_LAST_NAME_NN	Check	"LAST_NAME" IS NOT NULL	(null)
9 EMP_MANAGER_FK	Foreign_Key	(null)	HR
10 EMP_SALARY_MIN	Check	salary > 0	(null)

Oracle SQL / GUI 를 통해서 본 테이블의 제약조건

Oracle SQL Developer에서 주로 쓰이는 제약조건의 종류와 그 특성을 요약하면 다음과 같습니다.

Constraints	NULL 허용 여부	데이터 중복 허용여부	특징
NOT NULL	NULL 불가	중복 가능	
UNIQUE	NULL 가능	중복 불가 (* NULL끼리는 중복으로 간주하지 않음)	
PRIMARY KEY(고유 키)	NULL 불가	중복 불가	지정한 열은 유일한 값을 반드시 가져야 함 테이블 당 1개만 지정가능
FOREIGN KEY(외래키)	다른 테이블 열을 참조하여 해당 테이블에 존재하는 값만 입력 가능 다른 테이블의 고유키(PRIMARY KEY)를 참조		
CHECK	설정된 조건식을 만족하는 데이터만 입력가능 조건식을 만족하지 않는 데이터는 입력이 거부됨		

제약조건은 변수별로 설정되어 있습니다.

- Null이면 안 된다.
- 중복되지 않는 고유의 값이어야 한다.
- 다른 테이블로부터 참조해야 한다 / 참조할 데이터가 없으면 입력할 수 없다
- 미리 설정한 조건을 만족해야 한다

이렇게 컬럼에 설정된 조건을 제약조건(Constraints)이라고 합니다.

제약조건은 테이블을 생성할 때 함께 설정할 수 있고, 추후에 ALTER 등의 명령어를 사용하여 바꿀 수도 있습니다.

- 이미 데이터가 포함되어 있는 상황에서 제약조건을 함부로 바꾸기 어려우므로 사전에 테이블 설계를 잘 해 두는 편이 좋습니다.

ALTER를 통해 제약조건 수정

다시 한번 테이블을 만들어보자.

```
CREATE TABLE MEMBERS (  
MM_CODE NCHAR (5) ,  
MM_NAME NVARCHAR2 (5) ,  
MM_PASSWORD NVARCHAR2 (4) ,  
MM_PHONE NVARCHAR2 (11) ,  
);
```

에서

```
ALTER TABLE MEMBERS ADD CONSTRAINT MM_NAME_CK CHECK(LENGTH(MM_NAME) >= 2);
```

이렇게 적용 가능

```
CREATE TABLE MEMBERS(  
MM)CODE NCHAR (5),  
MM_NAME NVARCHAR2 (5),  
MM_  
);
```

ORACLE

MYSQL → 둘 다 AUTO COMMIT

테이블 특징

- 한 테이블내에서 칼럼명은 중복 X
- 칼럼 정의는 괄호 안에 기술 ()
- 각 컬럼은 ,로 구분
- 테이블명, 칼럼명은 숫자로 시작 X
- 세미콜론으로 끝

CREATE - Table 생성

CREATE TABLE ____ ();

칼럼명 데이터 타입 {DEFAULT 값} {제약조건},

**** DEFAULT, NOT NULL, NULL → 여부 생략 가능 NOT NULL 없으면 NULL 가능**

**** CHAR → 사이즈 만큼 빈자리는 공백으로 채움**

**** VARCHAR2 ORACLE(VARCHAR SQL) → 사이즈보다 문자 값 작아도 유지**

```
CREATE TABLE EXAM (  
  TEACHER_NO    NUMBER NOT NULL;  
  TEACHER_NAME  VARCHAR2(20) NOT NULL;  
  CONSTRAINT TEACHER_PK PRIMARY KEY (TEACHER_NO),  
  CONSTRAINT TEACHER_FK FOREIGN KEY (TEACHER_NAME) REFERENCE  
  CONSTRAINT TEACHER_NO_CK CHECK(TEACHER_NO>=18)  
);
```

REFERENCES {table 이름}{참조할 칼럼이름}

→ 참조할 칼럼 = 현재 table의 fk를 가져갈 table의 주식별자PK 속성 이름

ALTER - Table, column 변경 및 추가

- 칼럼 이름 변경 - RENAME
- 데이터 타입 변경 - MODIFY
- 사이즈 변경 - MODIFY
- DEFAULT 변경 - MODIFY

- 칼럼 삭제 - DROP
- 제약조건 추가, 삭제 - MODIFY

기본은

ALTER TABLE __테이블명__ → 으로 TABLE 가져와서 시작

ADD, MODIFY 는 바로 뒤에 COLUMN 안 붙이고 바로 정의

DROP, RENAME 은 바로 뒤에 COLUMN 붙임

```
ALTER TABLE 테이블명 ADD 칼럼명 데이터타입 {DEFAULT} {제약조건}
ALTER TABLE 테이블명 ADD CONSTRAINT 제약속성명 제약조건 (칼럼명)
```

- > NOT NULL 속성은 맨처음에 부여 불가능
- > 이미 TABLE에 여러 속성들 있을텐데 거기에 ADD 되는거
- > 모두 NULL로 채워지기 때문
- > BUT ! 만약 DEFAULT 선언해놓은 상태면 NOT NULL 같
- > DEFAULT 값으로 이미 NULL말고 다른 값들이 다 들어기

[칼럼 수정 ORACLE = MODIFY]

```
ALTER TABLE 테이블명 MODIFY 칼럼명 데이터타입 {DEFAULT} {제약조건}
ALTER TABLE 테이블명 MODIFY (칼럼명 데이터타입 {DEFAULT} {제약조건})
```

- > () 괄호 붙여도 되고 안붙여도 된다.
- > MODIFY는 동시에 여러개 불가능하다.

[칼럼 수정 SQL SERVER = ALTER COLUMN]

```
ALTER TABLE 테이블명 ALTER COLUMN 칼럼명 데이터타입 {DEFAULT} {제약조건}
```

- > ALTER COLUMN 은 () 사용하지 않고
- > 여러개 동시에 수정 불가능하다.

(기존에 있던 칼럼을 KEY로 만들 수도 있음)

```
ALTER TABLE 테이블명1 ADD CONSTRAINT KEY이름 FOREIGN KEY
REFERENCES 테이블명2(지정칼럼)
```

- > KEY 지정할 땐 칼럼에 무조건 () 씌워줘야한다.

- > ADD, MODIFY 둘 다 명령어 뒤에 COLUMN 이 오지 않음
- > MODIFY로 데이터 크기 줄이는건 안됨, 그러나 늘리는건

- > 데이터 타입도 바꾸려면 안에 들어있는 데이터가 없어야함
- > NULL값이 칼럼에 없어야 NOT NULL 제약 조건이 추가 가능

```
ALTER TABLE 테이블명 DROP COLUMN 칼럼명;
ALTER TABLE 테이블명 RENAME COLUMN 기존 칼럼명 TO 변경할 칼럼명;
```

- > DROP, RENAME은 COLUMN 이 붙어야한다.
- > 왜냐 !? DROP, RENAME은 COLUMN 말고 TABLE에도 가능

- 칼럼 순서는 변경 불가능 → 지우고 순서대로 처음부터 만들어야함

DROP - Table 삭제

DELETE, DROP, TRUNCATE 차이점

- DELETE : 데이터 일부 또는 전체 삭제(DML이니깐) → 롤백 가능
- DROP : 데이터와 구조를 동시 삭제, 즉시 반영(DDL이니깐) → 롤백 불가능
- TRUNCATE : 데이터만 초기화, 구조는 가만히 둬, 즉시 반영(DDL) → 롤백 불가능

```
DROP TABLE 테이블명;
DROP TABLE 테이블명 CASCADE CONSTRAINT;
```

- > 해당 테이블에 FK가 존재해서 다른 엔티티를 참조중이면 DROP 불가
- > CASCADE CONSTRAINT = 참조 제약조건을 모두 삭제하면서 동시에

▼ 제약조건에 따른 동작(Referential Action)

동작 방법 CONSTRAINT; 로 온다.

제약 조건 지정 없는데 삭제하려하면 에러 발생한다.

- CASCADE : Master 삭제시 child 같이 삭제
- Set NULL : Master 삭제시 child 필드 NULL
- Set DEFAULT : Master 삭제시 child 필드 DEFAULT
- RESTRICT : CHILD 테이블에 PK 값 없으면 MASTER 삭제 허용(에러 발생)
- **AUTOMATIC** : MASTER 테이블에 PK 없으면 MASTER PK 생성 후 CHILD 입력

- **DEPENDENT** : MASTER 테이블에 PK가 존재할 때만 CHILD 입력 허용

RENAME - 이름 변경

```
RENAME TABLE 변경 전 이름 TO 변경 후 이름
```

TRUNCATE - 테이블 값 초기화 (구조는 그대로, 복구는 안됨)

: DELETE, DROP과 다르게 구조는 남긴다. 값만 초기화 하는 것

```
TRUNCATE TABLE TEACHER;
```

▼ DML (Data Manipulation Language) → 데이터를 변형 **SQL AUTO**

Oracle DML = COMMIT 명령 직접 해야함

MYSQL DML = AUTO COMMIT

INSERT (삽입)

INSERT **INTO** 테이블명 칼럼명 VALUES 리스트

- 행단위 실행
- 한 번에 한 행만 입력 가능 → SQL은 여러 행 동시 삽입 가능
- NOT NULL 속성에 값 안들어가면 에러
- INTO (table) (column) 에서 칼럼명 명시 안되면, values로 모든 칼럼에 해당하는 값을 넣어야함
- VALUES 는 생략 불가능
- 칼럼 지정에 빠져서, 작성하지 않는 칼럼은 NULL 입력

UPDATE (수정)

UPDATE 테이블명 **SET** 칼럼명 = data WHERE

UPDATE 테이블명 (**SET** 칼럼명 = data , 칼럼명 = data, 칼럼명 = data)
WHERE 조건

- 칼럼 단위 실행

DELETE (삭제)

DELETE **FROM** ____ WHERE ____

- 행 단위 실행
- 별도의 COMMIT 없으면 롤백 가능

MERGE (병합)

INTO → 타겟 테이블명 : 애를 수정할 것

USING → 비교 테이블명 : 애를 참조해서 수정할 것

ON → 조건 : 이 조건이 TRUE, FALSE 이면 그에 따라 수정할 것

▼ TCL (Transaction Control Language) → 트랜잭션, **오라클 AUTO**

ORACLE = AUTO COMMIT

SQL = 안됨

트랜잭션이란?

- 논리적 연산 단위(하나의 업무)
- 분할 불가능한 업무의 최소 단위
- 하나의 트랜잭션엔 하나 이상의 SQL 문장 포함

트랜잭션 특징

- 고립성 → 각자 수행, 영향 X
- 지속성 → 수행후 Logging + Commit → 장애 발생해도 데이터는 문제 없이 유지
- 일관성 → 처리 전후 데이터의 합이 같아야함
- 원자성 → 모두 성공 or 모두 실패

COMMIT

ROLLBACK

: SQL에서 TCL은 테이블 SQL 별로 묵시적으로 동작하기 때문에 다음과 같은 메모리상 저장과 ROLLBACK에 의한 동작에 주의해야함

```
CREATE TABLE SAMPLE1 (COL1 NUMBER, COL2 NUMBER);
INSERT INTO SAMPLE1 VALUES(10,10);
CREATE TABLE SAMPLE2 (COL1 NUMBER, COL2 NUMBER);
INSERT INTO SAMPLE2 VALUES(10,30);
ROLLBACK; --> 여기서 롤백 발동
INSERT INTO SAMPLE2 VALUES(20,40);
COMMIT;
```

-> 이렇게 되면 CREATE 는 DDL 이므로 자동 AUTO COMMIT 이라서
그렇기 때문에 ROLLBACK 포인트 없이 그냥 ROLLBACK 하면 SAMPLE:
10, 30 넣었던 기억만 날아가고

SAMPLE1 에는 10, 10
SAMPLE2 에는 20, 40 들어가게된다.

SAVEPOINT

▼ DCL (Data Control Language) → 권한

USER 생성, 삭제

권한 부여, 회수

권한 목록

- CREATE SESSTION
- CREATE USER
- CREATE TABLE
- SELECT ON 테이블명
- ALTER ON 테이블명
- DELETE ON 테이블명
- INSERT ON 테이블명

**** 모든 권한을 줄 땐 ALL**

```
GRANT CREATE TABLE TO 사용자명;
GRANT SELECT ON 테이블명 TO 사용자명;
```

```
GRANT INSERT ON 테이블명 TO 사용자명;
```

```
REVOKE 권한명 FROM 사용자명;
```

ROLE 권한 부여, 회수

```
CREATE ROLE ROLE_1;
```

```
GRANT CREATE SESSION, CREATE USER, CREATE TABLE TO ROLE_1;
```

```
GRANT ROLE_1 TO 사용자명;
```

-> 해당 사용자는 ROLE_1에 저장된 모든 권한을 한번에 다 받을 수 있다.

▼ SQL 함수 (2과목 대비)

▼ 문자형 함수

- CHR(ASCII 코드)

: 코드 값에 따른 문자 출력

- LOWER(문자열) → 소문자

: 입력 문자열을 소문자로 변환

- UPPER(문자열) → 대문자

: 입력 문자열을 대문자로 변환

- LTRIM(문자열, [특정문자]) → 문자 사이에 있는 공백은 제거 안됨

: 왼쪽 공백 제거, 특정문자 제거(제거 되면 멈춤)

: 특정 문자로 여러 문자를 입력하면 그 안에 있는 모든 문자가 사라짐

ex) LTRIM('SQL', 'LE') → SQ

: 'LE' 에 'L', 'E' 가 있기 때문에 L, E를 날리는 것 대신 바로 멈춘다.

- RTRIM(문자열, [특정문자]) → 문자 사이에 있는 공백은 제거 안됨

: 오른쪽 공백 제거, 특정문자 제거(제거 되면 멈춤)

- TRIM([위치값] [특정문자] [FROM] 문자열) → , 없어도 됨 + 특정문자는 문자열 안됨

: 왼쪽, 오른쪽 공백 제거하고 특정 문자 제거

: 위치값은 **LEADING=왼쪽**부터, **TRAILING=오른쪽**부터, BOTH=둘 다

- **SUBSTR(문자열, 시작점, [길이])** → 0부터 시작 X, 1부터 시작임

: 문자열의 원하는 부분만 잘라서 반환(추출), 시작점 부터 자르기 시작해서, 길이 만큼 자른다.

: 더 잘라질 수 없으면 최대한의 값으로 출력

ex) SUBSTR('블랙핑크제니', 3, 2) → '핑크'

ex) SUBSTR('블랙핑크', 3, 3) → '핑크'

- **INSTR(문자열, 특정문자, [시작점], [몇번째에 발견])**

: 문자열에서 원하는 문자 찾아서 위치 반환, 여러개 찾자면 몇번째 문자의 위치를 반환

ex) INSTR('A#B#C#', '#', 3, 2) → 6

- **LENGTH(문자열)**

: 문자열의 길이를 반환

- **REPLACE(문자열, 찾는 문자열, [변경 할 문자열])**

: 문자열에서 특정 문자열을 찾아서 이를 변경 → **변경 할 문자 입력 안하면 없앴**

- **LPAD(문자열, 길이, 특정 문자)**

: 문자열이 설정한 길이가 될 때 까지 왼쪽을 **특정 문자로 채움**

- **RPAD(문자열, 길이, 특정 문자)**

: 위와 동일

- **CONCAT(문자, 문자) → 결합**

: 두 문자를 결합하는 함수

▼ 숫자형 함수

- **ABS(수)**

: 절댓값 반환

- **SIGN(수)**

: 부호를 반환 → 양수 = 1 / 음수 = -1 / 0 = 0

- **CEIL(수) → 올림**

: 소수점 이하의 수를 올림 한 정수로 반환

: 음수인 소수를 넣으면 버리게 되면 커진다.

ex) CEIL(72.86) → 73

ex) CEIL(-33.4) → -33

- **ROUND(수, [자릿수]) → 반올림**

: 수를 지정한 소수점 자릿수까지 반올림, Default = 정수로 만들 = 0

: 음수는 해당 자릿수의 정수를 반올림(-1 = 1의 자리를 반올림, -2 = 10자리)

ex) ROUND(163.76, 1) → 163.8

ex) ROUND(163.76, -2) → 200

- **TRUNC(수, [자릿수]) → 버림**

: 수를 지정한 소수점 자릿수까지 버림, Default = 0

: 음수는 해당 자릿수의 정수까지 버림

ex) TRUNC(54.29, 1) → 54.2

ex) TRUNC(54.29, -1) → 50

- **FLOOR(수) → 소수점 이하 버림**

: 소수점 이하의 수를 버림

ex) FLOOR(22.3) → 22

ex) FLOOR(-22.3) → -23

- **MOD(수1, 수2) → 나머지**

: 수1을 수2로 나눈 나머지를 반환

: 수2가 0일 경우 수1을 그대로 반환

ex) MOD(15, 7) → 1

ex) MOD(15, -4) → 3

▼ 날짜 함수

- **SYSDATE**

: 현재의 연, 월, 일, 시, 분, 초를 반환

: nls_date_format에 따라 출력 포맷 달라질 수 있음)

- **EXTRACT(특정 단위 FROM 날짜데이터 or SYSDATE)**

: 특정 단위의 날짜를 반환

ex) EXTRACT(YEAR FROM SYSDATE) → 2024

ex) EXTRACT(MONTH FROM SYSDATE) → 3

- ADD_MONTHS(날짜 데이터, 특정 개월 수)

: 입력한 날짜 데이터에 특정 개월 수를 더한 날짜를 반환해주는 함수

: 날짜의 이전 달이나 다음 달에 기준 날짜의 일자가 존재하지 않으면 해당 월의 마지막 일자 반환

ex) ADD_MONTHS(TO_DATE('2021-12-31'), 1) → 2022-01-31

ex) ADD_MONTHS(DATE '2022-01-31', 1) → 2022-02-28

▼ 변환 함수

명시적 형변환 : 변환 함수를 사용하여 데이터 유형을 명시적으로 나타냄

암시적 형변환 : DB가 내부적으로 알아서 데이터 유형을 변환함

- TO_NUMBER(문자열)

: 문자열을 숫자로 변환

: 숫자로 변환 안되는 진짜 문자가 들어가면 에러 발생

- TO_CHAR(수 or 날짜, [포맷])

: 수나 날짜 데이터를 문자형 또는 입력 포맷으로 변환

- TO_DATE(문자열, 포맷)

: 포맷 형식의 문자 데이터를 YYYY-MM-DD 형식의 날짜 데이터로 바꿈

▼ 그룹 함수 → 데이터들 모아서 처리함

그룹함수는 NULL이 들어간 행은 무시하고 처리한다.

- COUNT(대상)

: 행의 수 리턴

ex) COUNT(*) → * 입력하면 모든 칼럼에 대해 적용

- SUM(대상)

: 합을 리턴

- AVG(대상)

: 평균을 리턴

- MIN(대상)

: 최솟값 리턴

- MAX(대상)

: 최댓값 리턴

- VARIANCE(대상)

: 분산 리턴

- STDDEV(대상)

: 표준편차 리턴

▼ NULL 함수 & 치환 함수

- NVL(인수1, 인수2)

: 인수1의 값이 NULL일 경우 인수2를 반환

: NULL이 아니면 그대로 인수1 반환

- NULLIF(인수1, 인수2)

: 인수1과 인수2가 같으면 NULL 반환

: 같지 않으면 인수1을 반환

- COALESCE(인수1, 인수2, 인수3....)

: NULL이 아닌 최초의 인수를 반환

- NVL2(인수1, 인수2, 인수3) → 3개 까지

: 인수1이 NULL이 아니면 인수2, NULL이면 인수 3반환

ex) NVL2(REVIEW_SCORE, '리뷰있음', '리뷰없음')

함수가 아닌 CASE 구문형식으로도 특정 값을 치환 가능

- CASE 구문 & WHEN 조건 & ELSE 처리

: 별도의 ELSE가 없으면 NULL 값이 ELSE의 DEFAULT가 된다.

- DECODE(대상, 값1, 리턴1, 값2, 리턴2, 값3, 리턴3....., ELSE 값)

: CASE 구문과 같은 역할 조건의 구분은 없다.

▼ NULL - 함수의 NULL 처리

sum, avg, min, max, count 함수는

null을 무시한다.

SQL 함수 및 명령문은

맨위의 인스턴스부터 하나씩 인스턴스 훑는 순서로 진행된다

1 인스턴스 접근 → 1 인스턴스의 column에 접근 → 조건에 맞는 해당 인스턴스값 출력

2 인스턴스 접근 → 2 인스턴스의 column에 접근 → 조건에 맞는 해당 인스턴스값 출력

.....

COUNT() 해당 column의 행의 수를 출력

COUNT(*) → 모든 칼럼을 체크하여 행의 수를 출력

** 이 때 NULL이 포함된 column의 경우 카운팅 제외

NVL(column 명, value) → 지정 칼럼에서 NULL 찾아내고 value로 치환한 뒤 인스턴스를 출력